

RPC Part 1 Report

امیر حسین صباحی-810101556

امیر مهدی زریون-810100157

مبانی رایانش توزیع شده

1. هدف گزارش

در سیستم‌های توزیع‌شده، اجزای نرم‌افزار معمولاً روی ماشین‌ها یا سرویس‌های جدا اجرا می‌شوند و برای انجام یک کار مشترک باید از طریق شبکه با یکدیگر ارتباط داشته باشند. یکی از روش‌های مهم برای ساده کردن این ارتباط، استفاده از Remote Procedure Call یا RPC است. در RPC برنامه‌نویس به جای اینکه مستقیماً درگیر جزئیات ارسال پیام، ساخت packet، مدیریت socket و پردازش پاسخ شود، یک procedure یا method را صدا می‌زند. ظاهر کار شبیه فراخوانی یک تابع معمولی است، اما اجرای واقعی تابع ممکن است روی یک سرور دیگر انجام شود.

2. تعریف RPC و ایده اصلی

RPC مخفف Remote Procedure Call است. در این روش، یک برنامه در سمت client می‌تواند تابعی را که روی یک سیستم یا سرویس remote قرار دارد فراخوانی کند. هدف اصلی RPC این است که ارتباط بین سرویس‌ها برای برنامه‌نویس شبیه فراخوانی تابع عادی دیده شود. ایده اصلی این است که client درخواست خود را به صورت یک فراخوانی تابع می‌نویسد. سپس لایه RPC ورودی‌های تابع را به یک پیام قابل ارسال در شبکه تبدیل می‌کند، آن را به server می‌فرستد، پاسخ را دریافت می‌کند و نتیجه را دوباره به شکل مقدار خروجی تابع در اختیار client قرار می‌دهد.

3. تفاوت فراخوانی محلی و فراخوانی راه دور

فراخوانی محلی و فراخوانی راه دور از نظر ظاهر کد ممکن است شبیه باشند، اما از نظر اجراء وابستگی‌ها و نوع خطا تفاوت زیادی دارند.

معیار	فراخوانی محلی	فراخوانی راه دور
محل اجرا	تابع در همان process یا همان ماشین اجرا می‌شود.	تابع روی ماشین یا سرویس دیگری اجرا می‌شود.
وابستگی	معمولاً به شبکه وابسته نیست.	به network ، آدرس سرور و پروتکل ارتباطی وابسته است.
نوع خطا	خطاها بیشتر منطقی یا مربوط به خود برنامه هستند.	ممکن است timeout ، قطعی شبکه یا خرابی سرور رخ دهد.
کارایی	معمولاً سریع‌تر است.	معمولاً کندتر است، چون نیاز به ارسال شبکه‌ای و serialization دارد.
تغییرات	تغییر تابع معمولاً فقط در همان برنامه اثر دارد.	نیازمند مدیریت نسخه interface و سازگاری پیام‌هاست.

4. اجزای اصلی RPC

Client Stub و Server Stub

در RPC دو بخش مهم به نام Client Stub و Server Stub وجود دارد. Client Stub در سمت client قرار دارد و از دید برنامه‌نویس مثل همان تابع remote عمل می‌کند. وقتی برنامه client تابع را صدا می‌زند، در واقع Client Stub اجرا می‌شود.

Client Stub پارامترهای ورودی را جمع‌آوری می‌کند، آن‌ها را به پیام شبکه‌ای تبدیل می‌کند و درخواست را به server می‌فرستد. در سمت مقابل، Server Stub پیام دریافتی را باز می‌کند، پارامترها را استخراج می‌کند، تابع واقعی را روی server صدا می‌زند و نتیجه را دوباره به پیام پاسخ تبدیل می‌کند.

Deserialization و Serialization

Serialization یعنی تبدیل داده‌های داخل برنامه، مثل struct، object، عدد یا string، به یک قالب قابل انتقال در شبکه. برای مثال در JSON-RPC داده‌ها معمولاً به JSON تبدیل می‌شوند و در gRPC معمولاً از Protocol Buffers استفاده می‌شود.

Deserialization عملیات برعکس است؛ یعنی پیام دریافتی از شبکه دوباره به داده‌های قابل استفاده در برنامه تبدیل می‌شود. اگر serialization و deserialization درست انجام نشوند، client و server نمی‌توانند منظور یکدیگر را دقیق بفهمند.

مفهوم Interface Definition Language یا IDL

Interface Definition Language یا IDL زبانی برای تعریف رسمی interface سرویس است. در IDL مشخص می‌شود چه procedure هایی وجود دارند، ورودی هر procedure چیست و خروجی آن چه ساختاری دارد. این تعریف باعث می‌شود client و server یک قرارداد مشترک داشته باشند. در gRPC فایل proto نقش IDL را دارد. در Apache Thrift نیز فایل thrift برای تعریف سرویس و نوع داده‌ها استفاده می‌شود. داشتن IDL در پروژه‌های بزرگ مفید است، چون از روی آن می‌توان code های لازم برای زبان‌های مختلف را تولید کرد.

5. خطاهای رایج در RPC

با اینکه RPC از دید برنامه‌نویس شبیه فراخوانی تابع عادی دیده می‌شود، اما از نظر عملی به شبکه و سرویس remote وابسته است. به همین دلیل خطاهای خاصی در RPC وجود دارد که در فراخوانی محلی معمولاً دیده نمی‌شود.

- قطع بودن شبکه یا در دسترس نبودن route بین client و server
- خاموش بودن server یا crash کردن سرویس remote
- رخ دادن timeout به دلیل کند بودن شبکه یا شلوغ بودن server
- ارسال دوباره درخواست و خطر اجرای تکراری یک operation
- ناسازگاری نسخه interface بین client و server
- خطا در serialization یا deserialization داده‌ها
- مشکلات امنیتی مثل authentication ضعیف یا ارسال داده حساس بدون encryption

6. مقایسه REST با RPC

RPC و REST هر دو برای ارتباط بین سرویس‌ها استفاده می‌شوند، اما نگاه آن‌ها متفاوت است. در RPC تمرکز روی فراخوانی procedure یا method است. در REST تمرکز روی resource ها و عملیات استاندارد HTTP مثل GET ، POST ، PUT و DELETE است.

معیار	RPC	REST
سبک طراحی	Procedure-oriented	Resource-oriented
مدل ارتباط	معمولاً فراخوانی method/service	استفاده مستقیم از HTTP methods
فرمت داده	JSON, XML, Protocol Buffers یا قالب‌های دیگر	اغلب JSON یا XML
کاربرد رایج	مناسب برای ارتباطات داخلی و سرویس‌های دقیق	ساده و مناسب برای API های عمومی
مزیت مهم	در بعضی فناوری‌ها سریع‌تر و type-safe تر	معمولاً ساده‌تر برای browser و web API
Streaming	در gRPC پشتیبانی قوی از streaming وجود دارد.	Streaming معمولاً ذاتی نیست.

7. مقایسه فناوری های رایج RPC

در این بخش چند فناوری رایج RPC بررسی می‌شود. بیش از سه فناوری مقایسه شده اند تا تفاوت‌ها روشن تر باشد.

Use Case	Streaming	IDL	Data Format	Technology
Microservices و ارتباط داخلی سریع	Yes	Yes	Protocol Buffers	gRPC
Simple APIs و سرویس‌های سبک	No	No / Optional	JSON	JSON-RPC
Legacy Systems و سیستم‌های قدیمی	No	No	XML	XML-RPC
سیستم‌های چندزبانه و مقیاس‌پذیر	Limited / Yes	Yes	Binary / Compact / JSON	Apache Thrift
برنامه‌های Java-to-Java	No	Java Interface	Java Serialization	Java RMI
Enterprise و سرویس‌های قدیمی سازمانی	No	WSDL	XML	SOAP

معیارهای مقایسه

از نظر نوع داده و فرمت پیام، gRPC معمولاً از Protocol Buffers استفاده می‌کند که فشرده و سریع است. JSON-RPC ساده‌تر و خواناتر است، اما معمولاً سربار بیشتری نسبت به قالب‌های binary دارد. XML-RPC و SOAP به دلیل استفاده از XML پیام‌های بزرگ‌تری تولید می‌کنند. از نظر تعریف interface، gRPC و Apache Thrift به IDL وابسته‌اند و برای سیستم‌های بزرگ مناسب‌تر هستند.

JSON-RPC و XML-RPC ساده‌ترند و معمولاً بدون IDL رسمی هم قابل پیاده‌سازی هستند. Java RMI بیشتر مناسب محیط‌هایی است که client و server هر دو Java باشند. از نظر پشتیبانی چندزبانه، gRPC، JSON-RPC، XML-RPC، Apache Thrift و SOAP می‌توانند در زبان‌های مختلف استفاده شوند.

Java RMI محدودتر است، چون به اکوسیستم Java وابستگی بیشتری دارد. از نظر کارایی، gRPC و Apache Thrift معمولاً برای ارتباطات داخلی پرکاربرد و با سربار کمتر مناسب‌ترند. JSON-RPC برای پروژه‌های ساده و سبک مناسب است. XML-RPC و SOAP به دلیل استفاده از XML سربار بیشتری دارند. از نظر streaming نیز gRPC بهترین پشتیبانی را دارد.

8. پرسش‌ها

۱. چرا RPC باعث می‌شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد؟ زیرا جزئیات ارتباط شبکه‌ای پشت Stub ها پنهان می‌شود. برنامه‌نویس در سمت client فقط یک method را صدا می‌زند و ورودی و خروجی آن را می‌بیند Client Stub. درخواست را بسته‌بندی می‌کند، به server می‌فرستد و پاسخ را به شکل نتیجه تابع برمی‌گرداند.

۲. چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟

چون تابع `remote` واقعاً محلی نیست و به شبکه، زمان پاسخ، وضعیت `server` و سازگاری `interface` وابسته است. اگر برنامه‌نویس این موضوع را در نظر نگیرد، ممکن است `timeout`، خطای شبکه، اجرای تکراری درخواست یا کندی ارتباط را درست مدیریت نکند.

۳. چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

در فراخوانی راه دور ممکن است `server` در دسترس نباشد، شبکه قطع شود، پاسخ دیر برسد، `packet` ها گم شوند، درخواست دوباره ارسال شود، `client` و `server` نسخه‌های ناسازگار از `interface` داشته باشند یا داده‌ها هنگام `serialization` درست تبدیل نشوند.

۴. در چه شرایطی `gRPC` انتخاب بهتری از `REST` است؟

وقتی سیستم شامل `microservices` متعدد است، ارتباط داخلی بین سرویس‌ها زیاد است، کارایی و `latency` اهمیت دارد، `type-safety` مهم است، نیاز به قرارداد رسمی `interface` داریم یا `streaming` لازم است، `gRPC` معمولاً انتخاب بهتری از `REST` است.

۵. در چه شرایطی `REST` انتخاب ساده‌تر یا مناسب‌تر است؟

وقتی `API` برای استفاده عمومی یا مستقیم توسط `web client` طراحی می‌شود، سادگی پیاده‌سازی مهم‌تر از کارایی حداکثری است، عملیات‌ها `resource-oriented` هستند یا تیم می‌خواهد از ابزارهای ساده `HTTP` و `JSON` استفاده کند، `REST` انتخاب ساده‌تر و مناسب‌تری است.